Explainable AI-Driven Autonomous Self-Healing Cloud Management Platform

Rahul TS
Department of Computer Science and Engineering
School of Computer Science and Technology
Karunya Institute of Technology and Science
Coimbatore, Tamil Nadu, India
rahult23@karunya.edu.in

Abstract—Cloud infrastructure management increasingly depends on automated systems capable of detecting resource failures, predicting their onset before critical impact, and selecting appropriate recovery actions without continuous human intervention. This paper presents an explainable AI-driven autonomous cloud management platform that integrates ten sequentially connected processing stages: telemetry ingestion, anomaly detection, temporal buffer management, temporal feature extraction, multi-model failure-risk prediction, deterministic model-coefficient attribution, likely-failure-category inference, autonomous decision logic, simulated recovery, and post-recovery verification. The system is implemented as a deployable research prototype and evaluated on a controlled synthetic telemetry dataset covering eight distinct scenario profiles: two normal baseline profiles (stable and variable) and six failure scenarios (CPU spike, gradual CPU failure, memory leak, disk gradual exhaustion, network degradation, and combined gradual failure). Isolation Forest anomaly detection achieves precision 0.704, recall 0.826, and F1 0.760 on snapshot telemetry. Temporal Logistic Regression achieves mean F1 0.692 (± 0.036) and ROC-AUC 0.813 (± 0.021) across five random seeds, substantially outperforming both a majority-class baseline (F1 0.514) and a current-value threshold baseline (F1 0.156). Among 18 evaluated failure sequences, the model issued at least one high-risk prediction before labelled failure onset with a mean lead time of 18.06 timesteps. The XAI attribution layer computes exact coefficient-based contributions for each of 45 temporal features, enabling transparent identification of which telemetry patterns drive each prediction. A post-recovery verification mechanism re-evaluates simulated recovered telemetry using the same model, returning RECOVERED, NOT_RECOVERED, or INCONCLUSIVE verdicts. All experiments are conducted on synthetic data; real-world deployment validation is identified as future work.

Index Terms—cloud monitoring, anomaly detection, temporal feature engineering, explainable AI, coefficient attribution, AIOps, self-healing, failure prediction, autonomous remediation

I. Introduction

Modern cloud platforms continuously generate telemetry across CPU, memory, disk, and network resources. At scale, the volume of this telemetry far exceeds what human operators can monitor manually, and the time between failure onset and user-visible impact is often shorter than human response latency [1]. These pressures have driven adoption of Artificial Intelligence for IT Operations (AIOps) approaches that automate detection, diagnosis, and, increasingly, remediation of cloud infrastructure faults [2], [3].

The dominant body of work in cloud anomaly detection addresses individual pipeline stages. Detection methods — including Isolation Forest, LSTM-based log analysis [4], [5], and variational autoencoders [6], [7] — achieve high accuracy on specific telemetry types but do not connect their outputs to remediation. Self-healing systems [8], [9] trigger recovery actions from threshold events but typically lack temporal prediction or explainability. Root-cause analysis methods [10]–[12] recover causal structure from distributed traces or dependency graphs but require instrumented infrastructure and do not include verification of recovery effectiveness.

The result is a fragmented landscape: capable individual components that are rarely evaluated as an integrated end-to-end workflow from telemetry ingestion through post-recovery verification. This paper addresses that gap by designing, implementing, and evaluating a prototype that connects all stages of the cloud management lifecycle as a single evaluated system.

II. Related Work and Research Gap

A. Cloud Anomaly Detection

Early work on cloud anomaly detection applied statistical and unsupervised methods to snapshot telemetry. He et al. [13] systematically evaluated six classical ML models on system log data, finding that simpler models achieved competitive performance on specific datasets. Xu et al. [7] introduced Donut, a variational autoencoder for seasonal KPI anomaly detection in web services, achieving F1 scores of 0.75 to 0.90. Su et al. [6] proposed OmniAnomaly, a stochastic RNN applied to multivariate server metrics, achieving 0.89 F1 on the Server Machine Dataset. Similarly, Li et al. [14] proposed InterFusion, simultaneously modelling inter-metric and temporal dependencies for multivariate anomaly detection. These approaches demonstrate strong detection capability but output only anomaly scores without explaining which metrics or patterns triggered the alert.

Sequential log-based methods extend detection to event sequences. Du et al. [4] proposed DeepLog, an LSTM that

models normal log key sequences and detects deviations with F1 0.96 on HDFS logs. Meng et al. [5] introduced LogAnomaly, which combines semantic log templates with an LSTM classifier, achieving 0.98 F1 on BGL logs. Nedelkoski et al. [15] applied LSTM classification to distributed traces in a microservice testbed with F1 0.95. A shared limitation among these detection-focused systems is their inability to explain which features or log patterns drive the anomaly score and to map these detections to concrete remediation actions.

B. AIOps and Intelligent Operations

Lin et al. [1] surveyed the AIOps landscape at Microsoft Azure, identifying anomaly detection, root cause analysis, and automated remediation as the three central challenges. Bogatinovski et al. [2] provided a structured taxonomy of AIOps techniques for cloud platforms, covering metric, log, and trace modalities. Notaro et al. [3] surveyed AIOps failure management, categorising methods into detection, diagnosis, and automated remediation streams. Aceto et al. [16] conducted a comprehensive survey of cloud monitoring systems, observing that most tools address anomaly detection but do not integrate predictive modelling or autonomous recovery.

C. Explainability for Operational Systems

Ribeiro et al. [17] introduced LIME, which explains individual classifier predictions using locally fitted linear surrogate models. Lundberg and Lee [18] introduced SHAP, a game-theory-grounded approach that assigns feature contributions consistent with Shapley values. Both methods apply to arbitrary model types but introduce approximation. For Logistic Regression, direct coefficient-based attribution computes contribution as (scaled feature value $\times$ coefficient), which is mathematically exact and computationally inexpensive. Li et al. [19] surveyed XAI methods for anomaly detection, categorising explanations as feature-based, model-based, and post-hoc. This study leverages model-based attribution, which is applicable because the integrated system employs Logistic Regression.

D. Root Cause Analysis

Wang et al. [10] proposed CloudRanger, which constructs dynamic causal graphs from microservice dependencies and uses random walk to identify the root-cause service. Chen et al. [12] proposed CauseInfer, a hierarchical causal inference method operating on performance metrics, correctly identifying root cause services in 82% of cases. Weng et al. [11] applied the PC algorithm for causal discovery from cloud metric time series. These approaches require instrumented distributed tracing and explicit service dependency models. The present work uses a lighter-weight heuristic approach: XAI attribution strengths are aggregated by resource domain (CPU, Memory, Disk, Network) to infer the likely failure category. This does not establish causal proof but requires no tracing infrastructure.

E. Self-Healing and Automated Remediation

Samir et al. [8] proposed a fuzzy-logic self-healing system for Docker Swarm microservices, reducing failure recovery time by 60% compared with manual intervention, but relying on predefined rules without prediction or explainability. Gan et al. [20] proposed Seer, which predicts QoS violations in microservices from hardware telemetry with 91% accuracy, enabling preemptive resource allocation, but without XAI or RCA. Jiang et al. [9] demonstrated automated incident mitigation for cloud services by recommending troubleshooting guides, but without model-based explanation or post-recovery verification using the same model.

F. Research Gap and Open Challenges

The review in Section II reveals that the cloud management literature addresses individual pipeline stages well but rarely evaluates them as a unified end-to-end workflow. Specifically:

1) Detection methods do not explain which features drove the alert.
2) XAI methods are applied to detection models but not connected to recovery actions.
3) Self-healing systems do not use temporal prediction and do not verify recovery with the same model.
4) Survey papers on Artificial Intelligence for IT Operations (AIOps) repeatedly identify system integration as an open challenge, yet they rarely provide integrated experimental evaluations.

This work addresses these gaps by implementing and evaluating an integrated prototype that connects telemetry ingestion, temporal prediction, deterministic XAI attribution, failure-category inference, decision logic, simulated recovery, and model-based verification in a single system. While this system does not claim to be the first to combine cloud monitoring with self-healing, nor does it assert superiority over deep learning anomaly detectors, it provides a comprehensive end-to-end framework demonstrating a complete, interpretable, reproducible workflow in a research prototype setting.

G. Research Questions and Contributions

The specific research questions addressed are:

- RQ1: How effectively can ML-based anomaly detection identify cloud resource failures across diverse multi-scenario synthetic telemetry?
- RQ2: Can temporal rolling features improve early warning of gradual failures compared with snapshot threshold monitoring?
- RQ3: Can model-coefficient attribution identify which temporal telemetry patterns most influence each failure-risk prediction?
- RQ4: Can likely-failure-category inference and rule-based decision logic translate prediction outputs to appropriate recovery action selection?

- RQ5: Can post-recovery re-evaluation using the same predictive model provide a meaningful verification signal within an integrated self-healing workflow?

The main contributions of this work are:

1) An end-to-end integrated prototype connecting all ten cloud management stages in a single callable API.
2) A leak-free 45-feature temporal feature engineering layer using 10-step sliding windows.
3) A comparative predictive evaluation of five models across five random seeds with strictly sequence-isolated splits.
4) A deterministic logistic-regression coefficient attribution layer for XAI without approximation methods.
5) A post-recovery verification mechanism that re-evaluates recovery using the same trained model.

III. Proposed System Architecture and Methodology

The proposed methodology connects telemetry ingestion, anomaly detection, temporal feature extraction, multi-model failure-risk prediction, deterministic model-coefficient attribution, likely-failure-category inference, autonomous decision logic, and post-recovery verification as a unified sequential pipeline. The overall pipeline architecture is depicted in Fig. 1.

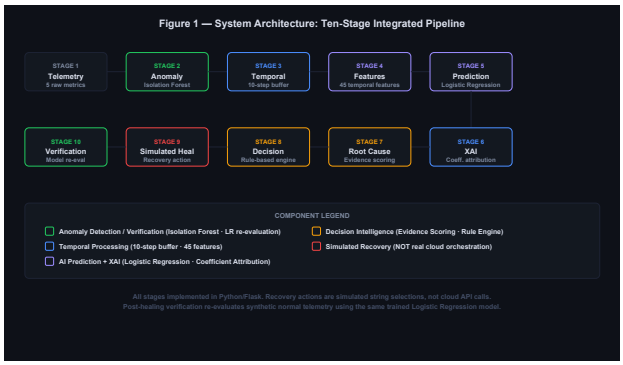


Fig. 1. Explainable AI-Driven Cloud Management System Architecture.

The proposed platform comprises ten processing stages arranged as a sequential pipeline:

1) Telemetry Ingestion: Synthetic generation of multi-metric sequences across CPU, memory, disk, and network interfaces.
2) Anomaly Detection: Snapshot point-in-time detection using an unsupervised Isolation Forest model.
3) Temporal Buffer Management: 10-step sliding window aggregation to accumulate historical temporal context.
4) Temporal Feature Extraction: Derivation of 45 leak-free statistical and regression features across trailing windows.
5) Failure Risk Prediction: Sequence-level classification using Logistic Regression to predict future failure probability.
6) XAI Attribution: Deterministic feature contribution computation via exact model coefficients and normalized values.
7) Likely Failure Category: Evidence score aggregation across resource domains (CPU, Memory, Disk, Network).
8) Autonomous Decision Logic: Rule-based selection mapping inferred failure category to targeted remediation.
9) Simulated Recovery: Mock execution of recovery logic (string selection representing remediation action).
10) Post-Recovery Verification: Model-based re-evaluation of simulated recovered telemetry using the same trained classifier.

A. Temporal Feature Engineering

A leak-free temporal feature set comprising 45 features is constructed from each 10-step trailing window. For each of the five raw metrics, nine features are computed: current value, 5-step mean, 10-step mean, 5-step delta, 10-step delta, 5-step slope (via linear regression), 10-step slope, 5-step standard deviation, and 10-step standard deviation. The feature extraction module strictly utilizes trailing observations, ensuring no access to future values (thereby preventing data leakage). StandardScaler is fit only on training rows and applied consistently during live inference.

B. Predictive Modeling and Target Formulation

Logistic Regression was selected for the integrated system with configuration: C=1.0, max_iter=2000, random_state=42, class_weight='balanced'. The prediction target is strictly forward-looking and binary: a label of 1 denotes that an active failure occurs within the subsequent 10 future timesteps ($t+1$ to $t+10$), strictly excluding the current observation at timestep t to avoid target circularity, and 0 otherwise.

The model is trained at server initialisation on the Phase 3C seed-42 training split (4,920 rows from 120 sequences). The fitted scaler and model are used for all live inference. Logistic Regression is selected over Random Forest and Gradient Boosting not because it achieves the highest aggregate performance — all three achieve similar F1 and AUC within measurement noise — but because its coefficient-based attribution is mathematically exact and requires no post-hoc approximation method.

C. Explainable AI and Failure Categorization

1) XAI Attribution: For each live prediction, the contribution of feature i is computed according to (1):

$$\text{contribution}_i = \text{scaled_value}_i \times \text{coefficient}_i \qquad (1)$$

where contribution$_i$ denotes the signed contribution of feature i to the predicted log-odds of failure, scaled_value$_i = (x_i - \mu_i)/\sigma_i$ is the z-score normalized feature value computed via the training-set StandardScaler, and coefficient$_i$ is the corresponding learned Logistic Regression weight. A positive contribution increases the predicted failure probability toward the high-risk threshold, whereas a negative contribution decreases it. The top 5 features by absolute contribution are reported for each prediction. This method is deterministic and computationally inexpensive.

2) Likely Failure Category Inference: Evidence scores for four resource categories (CPU, Memory, Disk, Network) are computed by summing absolute contributions of features belonging to each resource. The category with the highest summed evidence is reported as the likely failure category. This is a heuristic inference from model evidence, not a causal determination.

D. Decision and Simulated Self-Healing

When predicted failure risk reaches the actionable threshold, the autonomous decision engine maps the inferred likely failure category to a simulated recovery procedure, as summarized in Table I.

TABLE I
Likely Failure Category to Simulated Recovery Action Mapping

Likely Failure Category	Simulated Recovery Action
CPU Spike	Terminate / throttle high-CPU processes
Memory Leak	Restart affected service / reclaim memory
Disk Exhaustion	Clear logs / archive data / expand storage
Network Degradation	Check / restart network service / reroute
Combined Failure	Controlled recovery sequence + operator alert

E. Post-Healing Verification Protocol

When predicted risk reaches HIGH or CRITICAL:

1) Pre-healing risk (score, level, label) is recorded.
2) The decision engine selects a recovery action (simulated string selection, not executed via cloud API).
3) A 10-step synthetic normal_stable telemetry sequence is generated as a proxy for recovered state.
4) The same trained LR model evaluates this sequence.
5) Post-healing risk (score, level, label) is recorded.
6) Verification verdict determination:
   - RECOVERED: Post-healing risk level is evaluated as LOW (≤ 0.25).
   - NOT_RECOVERED: Post-healing risk level remains HIGH or CRITICAL (≥ 0.50).
   - INCONCLUSIVE: Post-healing risk level is evaluated as MEDIUM ($0.25 < \text{risk} < 0.50$), indicating partial remediation without achieving full baseline stability.

The temporal buffer is cleared after healing so the next polling cycle re-enters the WARMING_UP state.

IV. Experimental Setup

A. Dataset Configuration and Splits

The evaluation dataset encompasses eight distinct scenario types: two normal baseline profiles (normal_stable and normal_variable) and six failure scenarios (cpu_sudden_failure, cpu_gradual_failure, memory_gradual_leak, disk_gradual_exhaustion, network_degradation, and combined_gradual_failure). Across these eight scenarios, a total of 180 sequences were generated and partitioned into 120 training sequences (4,920 supervised rows), 30 validation sequences (1,230 supervised rows), and 30 test sequences (1,230 supervised rows), yielding exactly 7,380 supervised rows under a 10-step trailing window and 10-step prediction horizon censoring ($60 - 10 - 10 + 1 = 41$ rows per 60-step sequence), as summarized in Table II.

TABLE II
Dataset Splits and Supervised Row Counts

Split	Sequences	Supervised Rows
Training	120	4,920
Validation	30	1,230
Test	30	1,230
Total	180	7,380

Sequence-level splitting ensures no temporal window from a test sequence is visible during training. Zero sequence overlap is verified by an automated audit (evaluation_audit.json).

Positive label definition: A timestep t is labelled positive ($y_t = 1$) if an active failure is present in any of the next 10 future observations ($t + 1 \leq t' \leq t + 10$). The current observation at timestep t is strictly excluded from the target calculation, ensuring that the model is trained exclusively on advance predictive indicators rather than instantaneous state detection. Positive samples constitute approximately 52% of the dataset under this early-warning formulation.

B. Anomaly Detection Evaluation (Phase 2)

Isolation Forest is evaluated on snapshot telemetry. Baselines include a fixed-threshold classifier (alarm if any metric exceeds 80%) evaluated on identical data.

C. Temporal Predictive Evaluation (Phase 3)

Five models are compared across five random seeds {42, 123, 456, 789, 2026}:

1) Majority-class baseline
2) Current-value threshold baseline
3) Logistic Regression (C=1.0, max_iter=2000, class_weight='balanced')
4) Random Forest (n_estimators=100, class_weight='balanced')
5) Gradient Boosting (n_estimators=100)

Performance metrics include accuracy, precision, recall, F1, ROC-AUC, and PR-AUC.

D. Early-Warning Horizon Evaluation

Among test sequences containing labelled failure regions, the number of timesteps by which the model issues its first HIGH or CRITICAL prediction prior to the labelled failure onset is measured.

E. Explainability Evaluation Protocol

Four representative XAI case studies are documented from test sequences spanning different scenarios. For each case, the top 5 contributing features, their directions, and the evidence scores are recorded.

V. Experimental Results and Analysis

A. Detection Performance (Phase 2)

The baseline snapshot anomaly detection performance of Isolation Forest compared to the threshold baseline is shown in Table III.

TABLE III
Anomaly Detection: Isolation Forest vs Threshold Baseline

Method	Accuracy	Precision	Recall	F1	FPR
Threshold Baseline	0.903	1.000	0.435	0.606	0.000
Isolation Forest	0.910	0.704	0.826	0.760	0.073

The Isolation Forest achieves substantially higher recall than the threshold baseline (0.826 vs 0.435), correctly identifying most failure events at the cost of a 7.3% false positive rate. The threshold baseline produces zero false positives but misses 56.5% of failures — particularly gradual-onset scenarios where no metric exceeds 80% until late in the failure trajectory. Per-scenario detection analysis shows that CPU spike (F1 0.823), memory leak (F1 0.870), and network degradation (F1 1.00) are detected reliably. Disk gradual exhaustion achieves only F1 0.700 at snapshot level, reflecting the slow accumulation rate relative to the 80% threshold.

B. Temporal Predictive Performance (Phase 3)

The performance of the temporal predictive models aggregated over five independent random seeds is presented in Table IV and illustrated in Fig. 2.

TABLE IV
Temporal Model Comparison (5-seed mean ± std)

Model	F1	ROC-AUC	PR-AUC	Precision	Recall
Majority Baseline	0.514 ± 0.259	—	—	0.380 ± 0.193	0.800 ± 0.400
Threshold Baseline	0.156 ± 0.054	—	—	0.997 ± 0.006	0.086 ± 0.031
Logistic Regression	0.692 ± 0.036	0.813 ± 0.021	0.840 ± 0.027	0.730 ± 0.049	0.661 ± 0.050
Random Forest	0.696 ± 0.021	0.819 ± 0.005	0.847 ± 0.017	0.805 ± 0.047	0.620 ± 0.057
Gradient Boosting	0.699 ± 0.030	0.826 ± 0.013	0.852 ± 0.025	0.747 ± 0.137	0.701 ± 0.127

All three ML classifiers substantially outperform both baselines. The differences among LR, RF, and GB are small relative to within-seed variance. GB achieves the highest mean F1 (0.699) and AUC (0.826) but also the highest variance in recall (± 0.127). Logistic Regression achieves the most stable performance (F1 std ± 0.036). At seed 42 specifically: LR achieves F1 0.727, ROC-AUC 0.830; RF F1 0.718, ROC-AUC 0.819; GB F1 0.725, ROC-AUC 0.812.

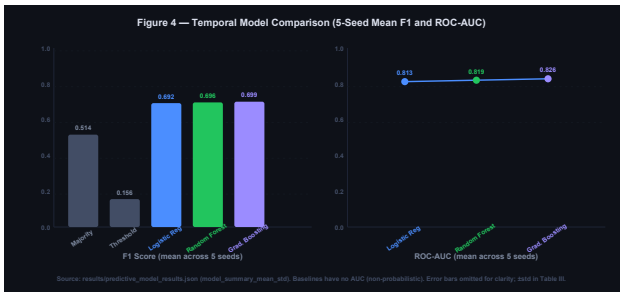


Fig. 2. Temporal Model Comparison (5-Seed Mean F1 and ROC-AUC).

Threshold baseline analysis: The current-value threshold achieves precision 0.997 but recall 0.086 — it correctly classifies near-zero events when a metric exceeds 80%, but misses the vast majority of failure timesteps because gradual failures build slowly below the threshold. This confirms that temporal feature-based prediction provides substantially better early-warning sensitivity.

C. Per-Scenario Predictive Performance

Table V reports the breakdown of classification performance across each evaluated failure scenario for Logistic Regression on the test split.

TABLE V
Per-Scenario Predictive Performance (LR, Seed 42, Test Set)

Scenario	Support	Prec	Recall	F1	TN	FP	FN	TP
normal_stable	205	0.00	0.00	0.00	132	73	0	0
normal_variable	164	0.00	0.00	0.00	164	0	0	0
cpu_sudden_fail	41	0.529	0.529	0.529	16	8	8	9
cpu_gradual_fail	164	0.817	0.527	0.641	41	13	52	58
mem_gradual_leak	41	1.000	0.649	0.787	4	0	13	24
disk_gradual_exh	246	0.952	0.791	0.864	12	9	47	178
network_degrade	123	0.635	0.783	0.702	36	27	13	47
combined_gradual	246	0.820	0.755	0.786	56	27	40	123

Several observations merit discussion:

- Normal scenarios produce zero true positives with moderate false positives (73/205 for normal_stable), reflecting that the model is sensitivity-oriented (class_weight='balanced'). In a production deployment, these false positives would manifest as false alarms.
- CPU sudden failure achieves the lowest F1 (0.529), consistent with the temporal prediction limitation: abrupt step changes produce limited trajectory signals before onset.
- Gradual failures (disk, memory, combined) achieve higher F1 (0.787–0.864), demonstrating that temporal features effectively capture accumulating resource patterns.
- Memory gradual leak achieves perfect precision (1.000), meaning all positive predictions on memory-leak sequences are correct, though recall (0.649) shows that early-trajectory timesteps are not yet captured.

D. Early-Warning Performance

The advance warning horizon achieved across all evaluated failure sequences is detailed in Table VI.

TABLE VI
Early-Warning Statistics

Metric	Value
Failure sequences evaluated	18
Mean lead time (timesteps)	18.06
Median lead time	16.0
Minimum lead time	4
Maximum lead time	33

The model issued at least one HIGH or CRITICAL prediction before the labelled failure onset in all 18 evaluated failure sequences. Mean lead time of 18.06 timesteps represents substantial advance warning. The minimum of 4 timesteps occurred on sudden-failure scenarios, consistent with the per-scenario results.

E. XAI Attribution Case Studies

Table VII summarizes representative XAI attribution case studies, with corresponding feature contributions visualized in Fig. 3.

TABLE VII
Representative XAI Case Studies

Case	Scenario	Risk	Top Feature	Contrib	Dir	Likely Cat
test_1	disk_gradual	0.591	Mem 10-step avg	-0.999	↓	Combined
test_4	combined_gradual	0.537	CPU 10-step var	-0.422	↓	Combined
test_6	network_degrade	0.534	Mem 10-step var	+0.416	↑	Combined
test_15	cpu_sudden	0.686	Mem 10-step avg	+0.917	↑	Memory Leak

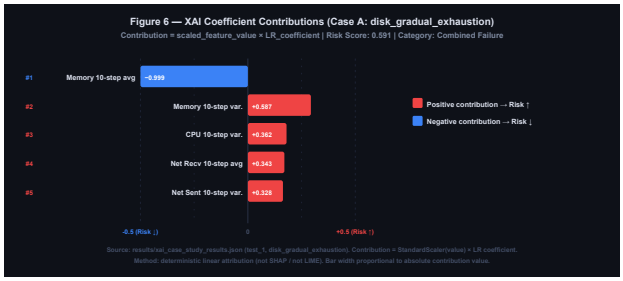


Fig. 3. XAI Feature Contributions Across Representative Evaluation Cases.

The XAI attribution shows that variability features (standard deviation) and slope features frequently dominate contributions in failure scenarios, reflecting that gradual failures increase temporal variation and trend steepness before absolute values reach alarm thresholds. Memory mean features show negative contributions in some cases, meaning that the model interprets high mean memory values as evidence against the specific failure label — a reasonable behaviour when the model has learned that combined failures show different multi-resource patterns.

F. Integrated Recovery Verification

The integrated system was evaluated across all six failure scenarios. For the network degradation scenario, the verification sequence and resulting risk transition are reported in Table VIII and depicted in Fig. 4:

TABLE VIII
Post-Recovery Verification (Network Degradation)

Stage	Value
Pre-healing risk	CRITICAL (0.884)
Simulated recovery	Controlled recovery sequence + alert
Post-healing risk	MEDIUM (0.447)
Verification verdict	INCONCLUSIVE

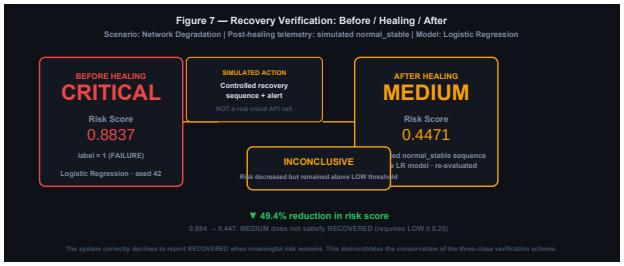


Fig. 4. Post-Recovery Verification Flow.

Risk decreased substantially (from 0.884 to 0.447) but did not reach LOW, producing INCONCLUSIVE. This is the expected and honest result: the model verifies against synthetic normal telemetry, and the specific synthetic sequence generated may include moderate variance that keeps the score above the LOW threshold. Healing actions are simulated string selections dispatched by the rule-based engine, not executable cloud orchestration API calls. Post-healing telemetry is synthetically generated normal telemetry used as an algorithmic evaluation proxy, not physical cloud recovery. The system correctly avoids falsely claiming RECOVERED.

VI. Discussion

A. Temporal Features vs. Threshold Detection

The results confirm that temporal rolling features substantially improve early-warning performance compared with snapshot threshold monitoring. The threshold baseline achieves recall of only 8.6% (mean across seeds) because gradual failures accumulate below any fixed threshold until late in the sequence. Logistic Regression with temporal features achieves recall of 66.1% (mean) with a mean lead time of 18 timesteps. This is the central empirical finding of the temporal component of this research.

B. Model Selection Trade-offs

Random Forest and Gradient Boosting achieve marginally higher mean F1 and AUC than Logistic Regression, but the differences are small relative to within-seed variance. The decision to integrate Logistic Regression is justified by interpretability: its coefficient-based attribution is exact, computationally inexpensive, and does not require SHAP or LIME approximation. This is a deliberate research design decision with a documented trade-off.

C. XAI for Operational Context

The coefficient attribution results show that variability and slope features are the strongest drivers of failure-risk predictions for gradual failure scenarios. This is operationally meaningful: an operator told that "CPU 5-step slope is the strongest positive contributor" can inspect the CPU trend chart and verify the attribution independently without relying on an unverified black-box approximation.

D. Root-Cause Inference Limitations

The heuristic for likely failure categories correctly identifies the dominant resource for most failure scenarios evaluated in the explainable artificial intelligence (XAI) case studies. However, it fails on sudden failures (where temporal features have not yet accumulated) and on combined failures (where evidence is distributed across all resources). These failure modes are documented and consistent with the broader temporal prediction limitations. The system does not claim causal diagnosis.

E. Post-Recovery Verification Conservatism

The INCONCLUSIVE verdict for network degradation is the honest and appropriate result. A system that claimed RECOVERED whenever risk decreased by any amount would overstate its recovery capability. The three-class verdict (RECOVERED / NOT_RECOVERED / INCONCLUSIVE) enforces a conservative stance consistent with academic integrity.

VII. System Limitations and Threats to Validity

1) Synthetic telemetry: All data is generated by parameterised scenario profiles, not from real production systems. Generalisation to real infrastructure is unvalidated.
2) Simulated recovery: Healing actions are string selections, not cloud API calls. No real resource reallocation occurs.
3) Sudden failure detection: Abrupt failures produce the lowest F1 (0.529) because temporal trajectory signals are limited before onset.
4) Root-cause inference is not causal proof: Evidence scores aggregate XAI contributions by resource domain; this is a heuristic mapping, not mechanistic causal discovery.
5) Post-healing telemetry is synthetic: The RECOVERED verdict reflects model behaviour on synthetic normal telemetry, not real system recovery.
6) Prototype model training at initialisation: The integrated system trains the LR model at Flask startup; this is not production-appropriate.
7) No real-world validation: Validation on real cloud trace datasets (Alibaba Cluster, Google Cluster, Azure) is future work.

VIII. Conclusion and Future Work

This paper presented an integrated, explainable AI-driven cloud management prototype that connects telemetry ingestion, Isolation Forest anomaly detection, temporal feature extraction, comparative predictive modeling, deterministic coefficient attribution, failure-category inference, autonomous decision logic, simulated recovery, and post-recovery model-based verification within a single evaluated system.

The key experimental findings are:

- Isolation Forest achieves F1 0.76 on snapshot anomaly detection.
- Temporal Logistic Regression achieves mean F1 0.692 and ROC-AUC 0.813 across five seeds, substantially outperforming threshold-based monitoring (F1 0.156).
- The model provides early-warning predictions with a mean lead time of 18.06 timesteps before labelled failure onset.
- Deterministic coefficient attribution identifies variability and slope features as the strongest predictive drivers for gradual failure scenarios.
- The post-recovery verification mechanism conservatively returns an INCONCLUSIVE verdict when recovery is partial, thereby preventing false claims of successful remediation.

Future work will address the validation of these methods on real cloud infrastructure trace datasets, the replacement of simulated recovery with real orchestration API integration, the evaluation of multi-model ensemble attribution methods, and rigorous comparisons with deep learning temporal methods (e.g., LSTM and transformer networks) under identical experimental protocols.

Acknowledgment

The author expresses sincere gratitude to project guide Mrs. S. Shirly, Assistant Professor, Department of Computer Science and Engineering, Karunya Institute of Technology and Science, for her invaluable supervision, guidance, and constructive feedback throughout this research. In compliance with IEEE author guidelines, the author acknowledges the use of AI tools for assistance with manuscript language refinement and formatting; all technical concepts, experimental analyses, and conclusions were independently conducted, verified, and approved by the author.

References

[1] Q. Lin, H. Zhang, J. Lou, Y. Zhang, and X. Liu, "AIOps: Real-world challenges and research innovations," in Proceedings of the 41st International Conference on Software Engineering: Companion (ICSE-Companion), 2019, pp. 324–325. [Online]. Available: https://ieeexplore.ieee.org/document/8802619

[2] J. Bogatinovski, S. Nedelkoski, J. Cardoso, and O. Kao, "Artificial intelligence for IT operations (AIOps) on cloud platforms: Reviews, trends and open challenges," IEEE Transactions on Cloud Computing, vol. 11, no. 4, pp. 3607–3624, 2023.

[3] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," ACM Transactions on Intelligent Systems and Technology, vol. 12, no. 6, pp. 1–45, 2021.

[4] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017, pp. 1285–1298. [Online]. Available: https://dl.acm.org/doi/10.1145/3133956.3134015

[5] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, and Y. Chen, "LogAnomaly: Unsupervised detection of sequential anomalies using deep learning," in Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI), 2019, pp. 4727–4734. [Online]. Available: https://www.ijcai.org/Proceedings/2019/658

[6] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2019, pp. 2828–2837. [Online]. Available: https://dl.acm.org/doi/10.1145/3292500.3330672

[7] H. Xu, W. Chen, N. Zhao, Z. Li, J. Bu, Z. Li, Y. Liu, Y. Zhao, and D. Pei, "Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications," in Proceedings of the 2018 World Wide Web Conference (WWW), 2018, pp. 187–196. [Online]. Available: https://dl.acm.org/doi/10.1145/3178876.3185996

[8] A. Samir, C. Dehury, and P. Harsh, "Self-healing cloud microservices using advanced fuzzy logic," IEEE Access, vol. 7, pp. 156 542–156 555, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/8880711

[9] J. Jiang, W. Lu, J. Chen, Q. Lin, P. Zhao, Y. Kang, H. Zhang, Y. Xiong, F. Gao, Z. Xu, Y. Dang, and D. Zhang, "How to mitigate the incident? an effective troubleshooting guide recommendation technique for online service systems," in Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2020, pp. 1410–1420.

[10] P. Wang, J. Xu, M. Ma, W. Lin, D. Pan, Y. Wang, and P. Chen, "CloudRanger: Root cause identification for cloud native systems," in Proceedings of the 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID), 2018, pp. 492–502. [Online]. Available: https://ieeexplore.ieee.org/document/8411019

[11] J. Weng, J. Wang, J. Yang, and Y. Yang, "Root cause analysis of anomalies of multitier services in public clouds," in Proceedings of the 26th International Symposium on Quality of Service (IWQoS), 2018, pp. 1–10. [Online]. Available: https://ieeexplore.ieee.org/document/8624177

[12] P. Chen, Y. Qi, P. Zheng, and D. Hou, "CauseInfer: Automatic and distributed performance diagnosis with hierarchical causality graph in large distributed systems," in Proceedings of the 33rd IEEE International Conference on Computer Communications (INFOCOM), 2014, pp. 1887–1895. [Online]. Available: https://ieeexplore.ieee.org/document/6848128

[13] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in Proceedings of the 27th IEEE International Symposium on Software Reliability Engineering (ISSRE), 2016, pp. 207–218. [Online]. Available: https://ieeexplore.ieee.org/document/7742964

[14] Z. Li, Y. Zhao, J. Han, Y. Su, R. Jiao, X. Wen, and D. Pei, "InterFusion: Interacting hidden dynamics and macroscopic observations for multivariate time series anomaly detection," in Proceedings of the 27th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2021, pp. 3172–3180. [Online]. Available: https://dl.acm.org/doi/10.1145/3447548.3467077

[15] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Cardoso, and O. Kao, "Anomaly detection and classification using distributed tracing and deep learning," in Proceedings of the 19th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID), 2019, pp. 241–250. [Online]. Available: https://ieeexplore.ieee.org/document/8752864

[16] G. Aceto, A. Botta, W. D. Donato, and A. Pescapè, "Cloud monitoring: A survey," Computer Networks, vol. 57, no. 9, pp. 2093–2115, 2013.

[17] M. T. Ribeiro, S. Singh, and C. Guestrin, ""why should I trust you?": Explaining the predictions of any classifier," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2016, pp. 1135–1144. [Online]. Available: https://dl.acm.org/doi/10.1145/2939672.2939778

[18] S. M. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 4765–4774. [Online]. Available: https://arxiv.org/abs/1705.07874

[19] Z. Li, Y. Shu, Y. Gong, and Y. Zhao, "A survey on explainable anomaly detection," 2023. [Online]. Available: https://arxiv.org/abs/2209.10398

[20] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, and C. Delimitrou, "Seer: Leveraging big data to navigate the complexity of cloud-based debugging," in Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019, pp. 19–32. [Online]. Available: https://dl.acm.org/doi/10.1145/3297858.3304004